Merci pour ces captures d'écran, elles sont parfaites pour le diagnostic !

Le mystère est résolu. Si tu regardes attentivement tes deux dernières images (notamment l'avant-dernière), on voit l'erreur exacte qui bloque tout :

👉 **pending reason: CreateContainerConfigError**

Qu'est-ce que ça veut dire ?

La bonne nouvelle, c'est que **ce n'est pas un problème de ressources** (CPU/RAM). Kubernetes a bien trouvé de la place sur un nœud pour ton pod. Par contre, au moment de construire le conteneur, Kubernetes a planté car **il lui manque un élément de configuration externe que tu lui as demandé d'injecter**.

Dans 99% des cas avec ce message d'erreur précis sur SparkaaS, la cause est : **Un Secret Kubernetes introuvable**.

La cause probable

Dans notre script spark-submit, on demande à Kubernetes d'aller chercher tes identifiants COS (AWS_ACCESS_KEY_ID et AWS_SECRET_ACCESS_KEY) dans un "Secret" Kubernetes pour les mettre en variables d'environnement.

Comme tu as changé de "Tenant" (tu es passé sur le namespace spark-a102567-dev-ef25bd0b), il y a deux possibilités :

1. Soit tu n'as pas remplacé la variable COS_SECRET_NAME dans le script bash que je t'ai donné, et Kubernetes cherche un secret avec un nom invalide.
2. Soit le secret avec tes identifiants COS **n'existe pas encore dans ce nouveau namespace**. Les secrets Kubernetes sont cloisonnés par namespace ; ce qui était dans ton ancien environnement de PoC n'est pas copié automatiquement dans le nouveau.

Comment régler ça tout de suite ?

Étape 1 : Confirme le diagnostic à 100% Dans ton terminal DevX, lance cette commande pour lire l'erreur détaillée :

Bash

```
kubectl describe pod poc-streaming-synchrone-test-26223e9cf76e8f45-driver -n spark-a102567-dev-ef25bd0b
```

Tout en bas, dans les *Events*, tu vas voir un message du style : Error: secret "NOM_DU_SECRET" not found. Cela te confirmera le nom exact du secret qui pose problème.

Étape 2 : Vérifie les secrets de ton nouveau namespace Regarde quels secrets sont réellement disponibles dans ton nouveau tenant avec cette commande :

Bash

```
kubectl get secrets -n spark-a102567-dev-ef25bd0b
```

Tu devrais y chercher un secret qui ressemble à s3-cos-... ou cos-credentials.

Étape 3 : Corriger

- **Si tu vois un secret COS dans la liste :** Mets simplement à jour la variable COS_SECRET_NAME="<le-vrai-nom>" dans ton script submit_streaming.sh et relance.
- **Si le secret n'y est pas :** C'est normal que ça bloque. Il faut recréer ce secret dans ton nouveau namespace spark-a102567-dev-ef25bd0b (soit via les process ITG Dataops, soit en le créant toi-même via kubectl create secret generic... si tu as les droits edit sur ce namespace, comme indiqué dans la page des droits MyAccess de ton PoC).

Dis-moi ce que te renvoie la commande kubectl get secrets !

Y a-t-il d'autres problèmes ?

En analysant tes captures d'écran en détail, **la réponse courte est non, il n'y a pas de "nouveau" problème majeur visible.** Ce que tu vois dans ton terminal, c'est simplement Kubernetes et Spark qui tournent en boucle. Comme le conteneur n'arrive pas à se créer à cause de la configuration manquante (CreateContainerConfigError), le client Spark attend et vérifie le statut toutes les secondes, d'où la répétition infinie des lignes Application status for spark-... (phase: Pending).

Cependant, on peut noter quelques détails intéressants dans tes logs pour te rassurer et anticiper la suite :

1. Les "Warnings" (Avertissements) normaux (à ignorer)

Sur ta première image, on voit deux messages au tout début de l'exécution :

- WARN NativeCodeLoader: Unable to load native-hadoop library... : C'est un grand classique de Spark. Il dit juste qu'il va utiliser les classes Java par défaut au lieu des bibliothèques natives C++ de Hadoop. C'est **totalement inoffensif** et normal dans un conteneur Kubernetes.
- INFO KerberosConfDriverFeatureStep: You have not specified a krb5.conf file... : Spark prévient qu'il n'a pas de configuration Kerberos (un système d'authentification). C'est **normal** car dans ton architecture, l'authentification se fait via les clés d'accès AWS/S3 du COS, pas par Kerberos.

2. Le point de vigilance : Le changement de "Namespace"

Puisque le problème vient d'une configuration introuvable (CreateContainerConfigError), il faut se rappeler d'une règle d'or de Kubernetes : **presque tout est cloisonné par "Namespace" (Tenant).**

Tu essaies de déployer sur le nouveau tenant spark-a102567-dev-ef25bd0b. Il est très probable que ce tenant soit "vide" de configurations par rapport à ton ancien environnement de PoC.

Voici les **3 éléments indispensables** que ton script spark-submit demande et qui doivent exister dans ce nouveau namespace :

1. **Le Secret COS** (s3-cos-...) : C'est ce dont on a parlé juste avant. Sans lui, le pod plante au démarrage.
2. **Le Service Account** (spark-sa) : On voit dans tes logs service account name: spark-sa. C'est l'identité de ton application. Si l'équipe ITG n'a pas créé ce compte dans le nouveau tenant, le pod n'aura pas les droits de faire son travail.
3. **Le Secret d'image** (all-icr-io) : Dans ton PoC, tu utilisais --conf spark.kubernetes.container.image.pullSecrets=all-icr-io pour avoir le droit de

télécharger l'image Docker Spark depuis le registre privé d'IBM. Ce secret doit aussi être présent ici !

La prochaine étape pour tout débloquer

Pour vérifier si ton nouveau tenant possède bien tous ces prérequis, lance ces deux commandes dans DevX :

Bash

```
# Vérifier les secrets (tu cherches s3-cos-... et all-icr-io)

kubectl get secrets -n spark-a102567-dev-ef25bd0b


# Vérifier les comptes de service (tu cherches spark-sa)

kubectl get sa -n spark-a102567-dev-ef25bd0b
```

Si l'un de ces éléments manque, tu as trouvé la cause exacte de ton CreateContainerConfigError ! Qu'est-ce que te renvoient ces commandes ?

J'ai amélioré l'architecture. Peux-tu me refaire le code en conséquence afin qu'il corresponde. Sachant que sparktacus c'est sparkaas

C'est une excellente évolution visuelle de ton architecture ! Le schéma avec les étapes numérotées est très clair et permet de bien comprendre l'objectif : avoir un service Spark qui tourne en continu et qui répond de manière synchrone à des requêtes HTTP (API REST).

Cependant, avant de te donner le code, je dois faire preuve de **candeur technique** sur ce modèle précis.

Spark est un moteur de calcul distribué conçu pour brasser des gigaoctets/téraoctets de données, pas un serveur web léger. Intégrer une API REST *directement* dans Spark pour traiter un "JSON unitaire" (une seule ligne) de manière synchrone est ce qu'on appelle un **anti-pattern** dans le monde de la Data :

1. **La latence :** Créer un DataFrame Spark, distribuer le calcul sur les exécuteurs et récupérer le résultat pour une seule ligne prendra plusieurs secondes (à cause de l'overhead de Spark), là où une API Python classique (FastAPI/Flask) prendrait quelques millisecondes.
2. **Le réseau Kubernetes :** Pour que ton DevX (l'API Client) puisse envoyer un POST à ton Pod Spark (l'API Server), il faudra que les équipes ITG exposent le port de ton Pod SparkaaS via un objet Kubernetes spécifique (un Service ou un Ingress), ce qui n'est pas natif avec un simple spark-submit.

Néanmoins, pour réaliser **exactement** le PoC que tu as modélisé sur ton schéma (étapes 3, 4, 5 et 6), la technique consiste à embarquer un mini-serveur web (comme Flask) directement à l'intérieur du *Driver* de ton application PySpark.

Voici le code et la méthode pour implémenter ton schéma.

1. Le code PySpark + API REST (Étape 3 et 5)

Ce script (spark_api_server.py) va initialiser Spark, puis démarrer un serveur web qui écoutera les appels API en permanence.

Note : Il faudra s'assurer que la librairie flask est bien présente dans ton environnement virtuel Python (venv-pack) que tu envoies au cluster, comme vu à la page 16 de ta documentation.

Python

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import concat_ws, lit, col

from flask import Flask, request, jsonify
```

```
import sys

# Initialisation du serveur web Flask
app = Flask(__name__)
spark = None

@app.route('/process', methods=['POST'])
def process_data():
    """
    Étape 4, 5 et 6 : Reçoit le JSON, le traite avec Spark, et renvoie le résultat.
    """
    try:
        # 1. Réception du JSON unitaire depuis l'API Client (DevX)
        input_data = request.json

        if not input_data:
            return jsonify({"error": "Aucune donnée fournie"}), 400

        # 2. Conversion du JSON en DataFrame Spark (calcul distribué)
        # Remarque : très coûteux en temps pour une seule ligne !
        df = spark.createDataFrame([input_data])

        # 3. Traitement Spark (Calcul, concat fictif)
        df_processed = df.withColumn(
            "unique_key", concat_ws("-", col("id_dossier"), lit("PROCESSED_VIA_API"))
        ).withColumn(
            "montant_double", col("montant") * 2
        )
```

```
        # 4. Récupération du résultat pour le renvoyer
        # On collecte la première (et unique) ligne et on la transforme en dictionnaire
        result_row = df_processed.collect()[0].asDict()

        # 5. Réponse synchrone (JSON unitaire)
        return jsonify(result_row), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500



def main():
    global spark

    # Étape 3 : Initialisation de l'application Spark
    print("Initialisation de la session Spark...")
    spark = SparkSession.builder \
        .appName("Sparktacus-REST-API") \
        .getOrCreate()

    print("Démarrage du serveur API REST sur le Driver Spark (Port 5000)...")
    # Fait tourner le serveur web en permanence et écoute sur toutes les interfaces
(0.0.0.0)
    app.run(host='0.0.0.0', port=5000)

if __name__ == '__main__':
    main()
```

---

2. Comment ça va se passer concrètement ?

En suivant les bulles numérotées de ton schéma :

- **Étape 1 & 2 (Spark Launcher) :** Depuis ton terminal DevX, tu lances ton script spark-submit habituel. Celui-ci déploie le Pod Driver et les Pods Executors sur SparkaaS (Sparktacus).
- **Étape 3 (Lancement Server) :** Le Pod Driver démarre, initialise la SparkSession, puis se bloque intentionnellement sur app.run(). Ton job apparaîtra en **RUNNING** dans la Spark UI, de manière permanente. Il écoute sur le port 5000.
- **Étape 4 (L'appel Client) :** Dans un autre terminal DevX (ou via un script Python/Postman), tu envoies ta donnée. C'est ici que réside le défi réseau : tu dois pointer vers l'IP interne du Pod Spark dans le cluster Kubernetes.

Exemple de requête :

Bash

```
curl -X POST http://<IP_DU_POD_DRIVER>:5000/process \
  -H "Content-Type: application/json" \
  -d '{"id_dossier": "DOS-999", "montant": 1500, "statut": "nouveau"}'
```

- **Étape 5 (Processing) :** Le serveur Flask reçoit la requête, donne l'ordre à Spark de créer le DataFrame, fait le .withColumn(), et rapatrie le résultat.
- **Étape 6 (Réponse) :** Le terminal DevX reçoit instantanément la réponse JSON modifiée : {"id_dossier": "DOS-999", "montant": 1500, "statut": "nouveau", "unique_key": "DOS-999-PROCESSED_VIA_API", "montant_double": 3000}.

La limite de cette architecture pour la production

Si ce modèle est génial pour prouver une architecture conceptuelle (PoC), en production, si 1000 fichiers arrivent en même temps (tes fameux "événements continus et concurrents"), ton serveur Flask hébergé sur le Driver Spark va s'effondrer sous les requêtes HTTP, car il n'est pas fait pour gérer une forte concurrence de la sorte, et Spark mettra trop de temps à allouer les tâches pour chaque JSON.

Pour le vrai streaming temps réel hautement concurrent, l'architecture standard consiste à ce que l'API "Client" pousse le JSON dans une file d'attente (comme Kafka) ou dans le COS, et que Spark lise cette file en *Structured Streaming* asynchrone (comme vu dans notre échange précédent).

du coup pour le code il faudrait que comme le précédent, il prenne des fausse données d'un test.csv et envoie un job dans le sparkui avec un processing en streaming et qu'il envoie une réponse synchrone

Et explique moi comment le lancer, le mettre en place, le faire fonctionner

Ton nouveau schéma est très clair et montre parfaitement la mécanique que tu souhaites mettre en place !

Tu as modélisé ce qu'on appelle une **architecture "Event-Driven" avec réponse synchrone**.

Cependant, je dois être transparent avec toi sur un point technique crucial : **Spark Structured Streaming (l'API native de streaming de Spark) n'est pas conçu pour renvoyer une réponse HTTP synchrone à un client.** Il est fait pour lire un flux en continu (ex: un dossier COS ou Kafka) et écrire le résultat ailleurs.

Pour réaliser **exactement** ce que tu as dessiné sur ton schéma (L'Application Source DevX envoie un JSON ➔ Spark le traite ➔ Spark renvoie le résultat JSON dans le terminal DevX), la solution consiste à **embarquer un mini-serveur Web (API REST) à l'intérieur du *Driver* Spark**.

Voici comment implémenter cette architecture de bout en bout pour ton PoC.

---

1. Le code "Spark Processing" (Le Serveur API sur SparkaaS)

Ce script (spark_serveur.py) sera envoyé sur ton cluster Kubernetes. Il va démarrer Spark, puis lancer un serveur Web (avec la librairie Flask) qui tournera en permanence en attendant tes données.

⚠️ *Prérequis : Il faudra t'assurer que la librairie flask est bien installée dans l'environnement virtuel Python (venv.tar.gz) que tu envoies à SparkaaS, comme expliqué page 16 de ta documentation.*

Python

```
# spark_serveur.py (À uploader sur ton COS)

from pyspark.sql import SparkSession

from pyspark.sql.functions import col, concat_ws, lit

from flask import Flask, request, jsonify

import logging
```

```
# Désactiver les logs Flask trop bavards
log = logging.getLogger('werkzeug')
log.setLevel(logging.ERROR)

app = Flask(__name__)
spark = None

@app.route('/process', methods=['POST'])
def process_data():
    """Endpoint qui reçoit le JSON, le traite avec Spark, et renvoie le résultat."""
    try:
        # 1. Réception de la ligne de données (JSON unitaire)
        input_data = request.json
        if not input_data:
            return jsonify({"error": "Aucune donnée reçue"}), 400

        # 2. Conversion en DataFrame Spark
        df = spark.createDataFrame([input_data])

        # 3. Traitements Spark (Calculs, concaténations, etc.)
        df_processed = df.withColumn("statut_traitement", lit("VALIDE_SPARK")) \
                        .withColumn("cle_unique", concat_ws("_", col("id_dossier"),
lit("PROCESSED")))

        # 4. Récupération du résultat (passage de Spark distribué vers le Driver)
        result_row = df_processed.collect()[0].asDict()

        # 5. Renvoi de la réponse synchrone au client
```

```
        return jsonify(result_row), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    print("Initialisation de la session Spark...")
    spark = SparkSession.builder \
        .appName("PoC-Spark-API-Synchrone") \
        .getOrCreate()

    print("Démarrage du serveur API REST sur le Driver Spark (Port 5000)...")
    # Le serveur tourne à l'infini et écoute les requêtes
    app.run(host='0.0.0.0', port=5000)
```

---

2. Le code "Application Source" (Le Client sur DevX)

Ce script (client_devx.py) va tourner localement sur ton terminal DevX. Il va lire ton fichier test.csv ligne par ligne, et envoyer chaque ligne à ton API Spark, comme un flux continu.

Crée un fichier test.csv simple dans le même dossier sur DevX :

Extrait de code

```
id_dossier,montant,client
DOS-001,1500,Alice
DOS-002,3200,Bob
DOS-003,450,Charlie
```

Puis, voici le code de l'application source :

Python

```
# client_devx.py (À exécuter localement sur DevX)
import csv
```

```
import requests
import time

# L'URL pointera vers le port local qu'on va rediriger vers le cluster
SPARK_API_URL = "http://127.0.0.1:5000/process"
FICHIER_CSV = "test.csv"

def lancer_flux():
    print(f"--- Début de l'envoi du flux depuis {FICHIER_CSV} ---")

    with open(FICHIER_CSV, mode='r', encoding='utf-8') as file:
        reader = csv.DictReader(file)

        for row in reader:
            print(f"\n[CLIENT] Envoi des données : {row}")

            # Envoi de la requête POST synchrone vers Spark
            response = requests.post(SPARK_API_URL, json=row)

            if response.status_code == 200:
                print(f"[SERVEUR] Réponse synchrone reçue : {response.json()}")
            else:
                print(f"[ERREUR] Code {response.status_code} : {response.text}")

            # Petite pause pour simuler un flux temps réel (streaming)
            time.sleep(1.5)

if __name__ == '__main__':
```

```
lancer_flux()
```

---

3. Comment lancer et faire fonctionner tout ça ? (Mode d'emploi)

Le gros défi de ton schéma, c'est le réseau : comment faire pour que l'API de DevX (ton client) puisse discuter avec l'IP interne du Pod Spark caché dans le cluster Kubernetes ? Voici la méthode de pro pour les PoC (le *Port-Forwarding*).

Étape 1 : Déploie le serveur Spark (Étape 1 & 2 de ton schéma)

Utilise le script submit_streaming.sh qu'on a vu précédemment pour lancer spark_serveur.py sur ton cluster (spark-a102567-dev-ef25bd0b).

Le job va se lancer. Il passera en RUNNING et ne s'arrêtera jamais. C'est normal, le serveur Web écoute.

Étape 2 : Récupère le nom du Pod Driver

Dans DevX, liste tes pods :

Bash

```
kubectl get pods -n spark-a102567-dev-ef25bd0b
```

Copie le nom du pod driver (ex: poc-spark-api-synchrone-...-driver).

Étape 3 : Créer le tunnel réseau (La magie s'opère ici)

Dans ton terminal DevX, tape cette commande pour relier le port 5000 de ton DevX au port 5000 de ton pod Spark :

Bash

```
kubectl port-forward pod/<NOM_DE_TON_POD_DRIVER> 5000:5000 -n spark-a102567-dev-ef25bd0b
```

(Laisse ce terminal ouvert, il maintient la connexion réseau).

Étape 4 : Lancer le flux de données (Étape 4, 5, 6 de ton schéma)

Ouvre un **deuxième** terminal DevX. Assure-toi que test.csv et client_devx.py sont dans le même dossier. Lance le script :

Bash

```
python client_devx.py
```

Ce que tu vas observer :

1. Dans ton terminal DevX, tu vas voir chaque ligne du CSV partir et revenir transformée instantanément avec les nouveaux champs ajoutés par Spark.

2. Si tu vas sur **l'Interface Spark UI** de ton tenant : tu verras l'application en statut RUNNING. Si tu cliques dessus et que tu vas dans l'onglet "Jobs", tu verras un nouveau mini-job Spark se créer et s'achever avec succès toutes les 1.5 secondes (à chaque fois qu'une ligne est envoyée par ton client).

C'est ce qui te permet d'avoir le beurre et l'argent du beurre pour ton PoC : le calcul Spark distribué **ET** la réponse synchrone dans le terminal.

Est-ce que cette approche te paraît claire à mettre en place sur ton environnement ?